Implementación de un simulador educativo de conducción térmica de bajo costo computacional

Grillas reducidas y visualización mediante OpenGL

Javier Armando Gutierrez

Departamento de Física, FCE, Universidad Nacional de Salta
Salta, Argentina
g.javier.ar@gmail.com

2 de noviembre de 2026
Informe técnico, revisión 2. Campañas del 28 y 29 de octubre de 2026

Resumen

Se presenta YoguiCuentas, un simulador educativo de conducción térmica bidimensional orientado a equipos de recursos limitados. Su diseño separa la grilla en la que se resuelve la ecuación de calor de la resolución de la imagen: OpenGL transforma un campo con pocos nodos en mapas y superficies animadas, sin exigir una incógnita térmica por píxel. Esto permite elegir la cantidad de nodos por las necesidades físicas del ejercicio y utilizar el renderizado para su presentación. El cálculo puede realizarse en CPU y conservar la aceleración gráfica disponible para visualizar, una opción pertinente para laboratorios sin GPU dedicada. Se describen la transferencia de campos, la interpolación y la publicación de estados confirmados, junto con un modelo conservativo con materiales, fuentes y contornos. Los métodos Euler, Crank–Nicolson, Jacobi y Gauss–Seidel permiten estudiar evolución y costo de refinamiento. Las campañas NumPy/OpenCL y Swift/Metal verifican balances y referencias analíticas; los tiempos registrados caracterizan el solver y no la tasa de cuadros de OpenGL. Se discute cuándo una grilla reducida resulta suficiente: la continuidad visual no añade resolución física ni corrige errores geométricos. El aporte es una herramienta de uso educativo que combina accesibilidad de hardware, visualización interactiva y una base física verificable.

Palabras clave: simulación educativa; conducción térmica; recursos limitados; renderizado OpenGL; grillas reducidas; visualización científica.

English title: Implementation of a computationally economical educational heat-conduction simulator: coarse grids and OpenGL visualization

Abstract. YoguiCuentas is an educational heat-conduction simulator for resource-constrained computers. It separates the numerical grid from image resolution: OpenGL renders a small set of nodal temperatures as maps and animated surfaces, without requiring one thermal unknown per pixel. CPU computation can be combined with available graphics hardware, supporting teaching laboratories without dedicated compute GPUs. We describe field transfer, interpolation and publication of confirmed states, together with the conservative thermal model. NumPy/OpenCL and Swift/Metal campaigns assess conservation and analytical references. Recorded timings characterize solver cost rather than OpenGL frame rate. Rendering does not add physical information or compensate for unresolved gradients and boundary errors. The contribution is a practical educational tool combining modest computational requirements, interactive visualization and a verifiable physical basis.

1. Introducción

Para trabajar con transferencia de calor en un laboratorio docente hace falta una herramienta que permita construir un problema, cambiar sus condiciones y observar qué ocurre. La posibilidad de explorar un material distinto, una fuente localizada o un contorno convectivo resulta especialmente útil cuando la respuesta puede verse durante la actividad. En ese contexto, el interés de la simulación incluye la facilidad de uso y la disponibilidad de los equipos donde se ejecuta, además de la precisión del cálculo.

Una imagen de alta resolución no necesita una grilla térmica con la misma cantidad de puntos. En problemas suaves, el campo puede representarse con un número moderado de nodos y dibujarse después sobre muchos más píxeles. Refinar la ecuación únicamente para que la imagen se vea continua introduce un costo evitable: aumenta las incógnitas y, en un integrador explícito, reduce el paso de tiempo permitido. El problema de diseño consiste en resolver la física con la grilla necesaria para el ejercicio y aprovechar el sistema gráfico para producir su representación visual.

YoguiCuentas [8] se desarrolla con ese propósito. Su edición Linux combina un solver CPU u OpenCL con renderizado OpenGL; permite editar geometrías, asignar materiales, fuentes y contornos y observar campos de temperatura y flujo. La visualización puede mantenerse aunque el cálculo térmico se ejecute en CPU. La edición macOS adapta el entorno a Swift/Metal y aporta otra implementación con la que contrastar los resultados numéricos. El eje de este artículo es la aplicación del renderizado a una herramienta educativa de bajos requisitos de hardware; las verificaciones fundamentan la física que esa herramienta muestra.

Se describe cómo se desacoplan resolución numérica y resolución de pantalla, qué partes del trabajo se mantienen proporcionales al número de nodos y qué puede esperarse de una grilla reducida. A continuación se documentan el modelo térmico, los métodos y los experimentos de comprobación. Finalmente se discuten los usos docentes, las condiciones de acceso a hardware y los límites de la estrategia. Se distingue la verificación numérica de la validación frente a mediciones físicas [5, 6]; esta última no forma parte de las campañas disponibles.

2. Diseño para equipos de recursos limitados

2.1. Resolver pocos nodos y dibujar muchos píxeles

El campo numérico contiene $N = n_x n_y$ temperaturas. La imagen ocupa $P = W\,H$ píxeles, donde W y H son las dimensiones del área de dibujo. Son cantidades independientes: aumentar el tamaño de la ventana cambia la rasterización sin crear nuevas incógnitas térmicas. En la caja rectangular completa, la malla de dibujo usa los nodos existentes y $2(n_x - 1)(n_y - 1)$ triángulos. OpenGL interpola los valores de sus vértices al generar los fragmentos y aplica el mapa de color mediante shaders [1].

Por ejemplo, una grilla 33×33 contiene 1089 temperaturas y genera 2048 triángulos en el caso rectangular. Puede representarse en un área de 1280×720 píxeles sin resolver 921600 ecuaciones térmicas. Esta comparación es una cuenta de tamaños ilustrativa, no una medición de velocidad. El trabajo gráfico sigue dependiendo de los píxeles, las capas y el dispositivo, pero ese trabajo no se convierte en otras tantas aplicaciones de la ecuación de calor.

Sobre cada triángulo plano se reconstruye visualmente

$$\widetilde{T}(\mathbf{x}) = \lambda_1(\mathbf{x})T_1 + \lambda_2(\mathbf{x})T_2 + \lambda_3(\mathbf{x})T_3, \qquad \sum_{a=1}^{3} \lambda_a = 1. \tag{1}$$

Los pesos corresponden a la interpolación del triángulo, no a una nueva solución del problema. En el rectángulo la reconstrucción es lineal por triángulos; no se la presenta como una interpolación bilineal exacta en toda la celda. Para recortes, el código construye abanicos sobre volúmenes duales

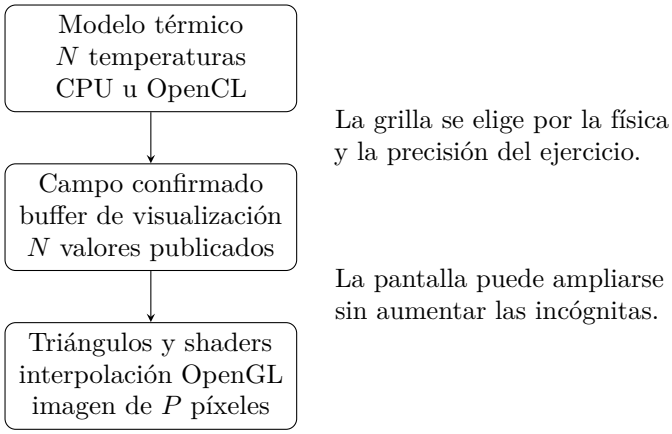


Figura 1: Separación entre cálculo y presentación en Linux. Esquema de la implementación; no representa un benchmark ni una captura de pantalla. El recorrido CPU conserva el renderizado OpenGL y el recorrido compartido OpenCL/OpenGL evita el tránsito del campo por el host cuando es compatible.

activos y combina únicamente nodos localmente conectados. Así evita dibujar una primitiva que atraviese una región excluida. La superficie 3D eleva esa malla según la temperatura; un número pequeño de vértices conserva su geometría facetada aunque el color resulte continuo.

2.2. Cálculo, transferencia y animación

En la ruta CPU, NumPy actualiza el campo térmico en un trabajador y publica una instantánea float32 en un buffer OpenGL. Este recorrido necesita un dispositivo gráfico compatible para dibujar, pero no una GPU dedicada para el cálculo térmico. Cuando OpenCL y OpenGL comparten contexto, la aplicación copia el campo calculado a un VBO compartido sin leerlo previamente en memoria del host. Se conserva una copia de publicación: compartir recursos no significa eliminar toda copia ni toda sincronización.

La propiedad del buffer se ordena mediante una secuencia de espera gráfica, adquisición por OpenCL, copia, liberación y dibujo. Si la interoperabilidad no es compatible se dispone de una ruta de transferencia por host. Cada imagen corresponde a un estado confirmado; el editor, el solver y la imagen visible no se confunden en un único campo mutable. Este diseño permite continuar dibujando la última solución disponible sin exigir que un paso físico coincida con cada cuadro de pantalla.

En la ruta CPU se limita la frecuencia de publicación aproximadamente a 30 actualizaciones por segundo. Es un límite de programación para evitar transferencias innecesarias, no una tasa de cuadros medida ni garantizada. La simulación puede requerir varios pasos entre publicaciones o tardar más que ese intervalo. El usuario puede pausar, cambiar la velocidad de reproducción e inspeccionar el estado sin interpretar el tiempo de pantalla como tiempo físico.

2.3. Qué significa bajo costo en este trabajo

El objetivo es reducir los recursos necesarios para una actividad concreta: cantidad de incógnitas, memoria del cálculo, trabajo por paso y necesidad de un acelerador dedicado. Un único campo float32 de 33^2 nodos ocupa 4356 bytes; uno de 257^2 ocupa 264196 bytes, unas 60,65 veces más. El programa mantiene otros buffers y consume memoria de interfaz y gráficos, por lo que estas cantidades no son la RAM total de la aplicación.

La economía principal proviene de no refinar la grilla para satisfacer una exigencia de presentación. Una vez elegida la resolución física, el renderizado aprovecha el hardware gráfico disponible

para mapas, superficies e indicadores. La estrategia es compatible con gráficos integrados si el controlador ofrece las funciones requeridas; ejecutar el solver en CPU no elimina el requisito de OpenGL para la interfaz Linux. La edición actual utiliza GLSL 1.40 y buffers de textura, por lo que la compatibilidad debe comprobarse en cada equipo.

Como antecedente de factibilidad se comunicaron pruebas preliminares de instalación y uso del cálculo CPU en equipos de laboratorio con Intel i3-3240/i5-4440 y 4/8 GB de RAM. Estas pruebas de uso motivan el diseño; no se dispone de un registro instrumentado de tasa de cuadros, memoria máxima y latencia para esos modelos. Las campañas formales que siguen utilizan Ryzen/A2000 y M1. Por ello se distinguen la posibilidad de uso informada y la caracterización cuantitativa del hardware de laboratorio, que queda por completar.

Una grilla reducida es adecuada cuando reproduce las escalas espaciales relevantes para el ejercicio. Puede mostrar difusión de un perfil suave o tendencias de un intercambio térmico sin necesitar resolución de imagen en el solver. Si hay una fuente estrecha, una interfaz mal representada o un gradiente intenso, corresponde refinar y comprobar el resultado. La continuidad del color ayuda a leer el campo; no crea información física entre nodos ni convierte una aproximación insuficiente en una solución precisa.

3. Modelo físico y discretización

3.1. Problema continuo y unidades

Para un dominio plano Ω, se considera

$$C(\mathbf{x})\frac{\partial T}{\partial t} = \nabla\cdot\left(k(\mathbf{x})\nabla T\right) + Q(\mathbf{x}), \qquad C = \rho c_p > 0, \quad k > 0. \tag{2}$$

T es temperatura; k, conductividad en $\mathrm{W\,m^{-1}\,K^{-1}}$; C, capacidad calorífica volumétrica en $\mathrm{J\,m^{-3}\,K^{-1}}$; y Q, potencia volumétrica en $\mathrm{W\,m^{-3}}$. Las propiedades no dependen de temperatura en el alcance ensayado. La representación bidimensional integra balances por unidad de espesor: la energía se expresa en J/m y la potencia en W/m. Las diferencias de temperatura se expresan en K aunque la interfaz presente temperaturas en grados Celsius.

En la frontera se admiten temperatura impuesta $T = T_b$, flujo saliente $-k\nabla T\cdot\mathbf{n} = q_N$ y convección $-k\nabla T\cdot\mathbf{n} = h_b(T - T_\infty)$, con $h_b \geq 0$. El caso adiabático corresponde a flujo nulo. Se excluyen radiación, advección, cambio de fase y propiedades dependientes de T; no se infiere su validez a partir de estos resultados.

3.2. Grilla, volúmenes duales y operador

La grilla es cartesiana nodal, uniforme en cada eje, con $\Delta x = L_x/(n_x - 1)$ y $\Delta y = L_y/(n_y - 1)$, que pueden diferir. Cada nodo activo posee un volumen dual V_i por unidad de espesor. Los volúmenes exteriores de la caja tienen anchuras reducidas y las máscaras determinan qué conexiones y caras expuestas participan. No se genera una malla ajustada a las curvas dibujadas.

En los grados de libertad libres, el balance puede escribirse

$$C_i V_i \dot{T}_i = \sum_{j\in\mathcal{N}(i)} G_{ij}(T_j - T_i) + Q_i V_i - \sum_{b\in\mathcal{B}(i)} P_{ib}, \qquad G_{ij} = \frac{k_{ij}a_{ij}}{d_{ij}}. \tag{3}$$

a_{ij} es longitud de cara por unidad de espesor y d_{ij}, distancia entre nodos. La conductividad de cara se obtiene por resistencias en serie; para semidistancias iguales resulta $k_{ij} = 2k_ik_j/(k_i + k_j)$. La simetría de la conductancia permite cancelar los intercambios internos en el balance total. Los nodos de temperatura impuesta no participan como incógnitas libres; sus reacciones térmicas deben considerarse al interpretar la energía global.

En un recorte interno, la pared se sitúa a media celda. La condición Robin combina resistencia conductiva y convectiva:

$$G_{ib}^{R} = \frac{a_{ib}}{d_{ib}/k_i + 1/h_b}, \qquad d_{ib} \in \{\Delta x/2, \Delta y/2\}. \tag{4}$$

Esta distancia no se debe atribuir indiscriminadamente a los nodos exteriores de la caja: la implementación distingue ambas representaciones. La temperatura Dirichlet de un recorte se impone mediante la conductancia a su cara; en la caja se fijan nodos exteriores. Las esquinas con dos temperaturas impuestas conservan la convención de promedio implementada.

El ensamblado conduce a

$$\dot{\mathbf{T}} = \mathbf{f} - A\mathbf{T}, \qquad r(\mathbf{T}) = \mathbf{f} - A\mathbf{T}, \tag{5}$$

con residuo r_i en K/s. El stencil es de cinco puntos; las pérdidas de contorno contribuyen a la diagonal y al forzamiento. En regímenes estacionarios se requiere una referencia térmica en cada componente libre conectada. Las componentes puramente Neumann/adiabáticas se rechazan en el modo estacionario actual: no se implementó un tratamiento general del espacio nulo con restricción de media.

Las escenas v2 representan fuentes de área con muestreo subcelda 8×8 y fuentes de línea con potencia por longitud. Esa cuadratura es una decisión de rasterización, no una reconstrucción exacta de la geometría. Las regiones activas contiguas conservan conducción entre materiales; una exclusión crea caras expuestas a las que se asignan condiciones de contorno.

4. Métodos y controles de la solución

4.1. Integración temporal y resolución estacionaria

Euler explícito usa $\mathbf{T}^{n+1} = \mathbf{T}^n + \Delta t\, r(\mathbf{T}^n)$. Para la estructura difusiva con coeficientes no negativos se limita el paso mediante la diagonal de pérdidas, con un factor de seguridad. En el caso rectangular homogéneo sin pérdidas adicionales se recupera la restricción $\alpha\Delta t(\Delta x^{-2}+\Delta y^{-2}) \le 1/2$, con $\alpha = k/C$. Un mayor número de nodos reduce el paso permitido.

Crank–Nicolson (CN) [2] resuelve

$$(I + \Delta t A/2)\mathbf{T}^{n+1} = (I - \Delta t A/2)\mathbf{T}^n + \Delta t\mathbf{f}. \tag{6}$$

La implementación utiliza incrementos $\delta = \mathbf{T}^{n+1} - \mathbf{T}^n$:

$$(I + \theta\Delta t A)\delta = \Delta t\, r(\mathbf{T}^n), \tag{7}$$

con $\theta = 1/2$ en CN. El sistema se resuelve por Jacobi ponderado, $\omega = 2/3$, y se controla el residuo del sistema de incrementos en K. No se reemplaza este solver interno por Gauss–Seidel al seleccionar ese método para el estacionario. Si un sistema no converge se rechaza el lote; no se publica como solución un campo parcial.

El arranque opcional de Rannacher reemplaza el primer paso físico por dos pasos de Euler implícito de longitud $\Delta t/2$, con $\theta = 1$, y luego continúa con CN. Su finalidad es amortiguar componentes de alta frecuencia de datos poco regulares [3]. Para $\dot{u} = -\lambda u$, $z = \lambda\Delta t$, se obtienen

$$g_E = 1 - z, \qquad g_{CN} = \frac{1 - z/2}{1 + z/2}, \qquad g_R = (1 + z/2)^{-2}. \tag{8}$$

La estabilidad de CN no asegura positividad ni ausencia de cambios de signo: con $z = 4$, $g_{CN} = -1/3$ y el arranque tiene $g_R = 1/9$.

Cuadro 1: Criterios disponibles. S es la selección de nodos y $m = |S|$; índices i, j en las dos medidas matriciales corresponden a fila y columna.

Criterio	Medida o condición
Residuo físico	Ecuaciones (10), siempre en todos los nodos libres.
ΔT L1	$\sum_S \lvert D_i\rvert$.
Euclídea/Frobenius	$\sqrt{\sum_S D_i^2}$.
RMS	$\sqrt{\sum_S D_i^2/m}$.
Máximo nodal	$\max_S \lvert D_i\rvert$, igual a la norma infinito del vector.
Norma matricial 1	$\max_j \sum_i \lvert D_{ij}\rvert$.
Norma matricial infinito	$\max_i \sum_j \lvert D_{ij}\rvert$, distinta del máximo nodal.
Traza/suma diagonal	$\lvert \sum_i D_{ii}\rvert$ sobre diagonal elegible.
Cambio de media	$\lvert \sum_S D_i/m\rvert$.

Para el estacionario $A\mathbf{T} = \mathbf{f}$ se ofrece Jacobi ponderado y Gauss–Seidel (GS) rojo–negro con $\omega = 1$. Jacobi actualiza simultáneamente a partir del campo anterior. GS colorea por la paridad de los índices, actualiza rojo y luego negro, con despachos ordenados; una iteración completa incluye ambos colores. La ordenación cambia los iterados frente a GS lexicográfico. Ambos son métodos iterativos [4]; el rótulo de interfaz "estacionario directo" significa ausencia de integración temporal, no factorización directa. Sus CSV cuentan iteraciones y no les asignan tiempo físico.

4.2. Residuo físico y criterios pedagógicos de parada

El criterio predeterminado exige simultáneamente

$$R_\infty = \max_{i\in F} |r_i| \le a_r + \varepsilon_r R_\infty^{(0)}, \tag{9}$$

$$B_1 = \sum_{i\in F} C_i V_i |r_i| \le a_r \sum_{i\in F} C_i V_i + \varepsilon_r B_1^{(0)}, \tag{10}$$

donde F contiene nodos libres activos. B_1 tiene unidades W/m y es una suma absoluta local: no es el balance neto con signo que podría anularse por cancelación. Las tolerancias generales predeterminadas son $a_r = 10^{-5}$ K/s y $\varepsilon_r = 10^{-5}$, con controles cada 128 pasos y tres confirmaciones consecutivas. Los ensayos pueden sobrescribirlas y registran sus valores; no se presentan los defaults como tolerancias universales de exactitud.

Para los criterios de cambio se define $D = T^{(c)} - T^{(c-1)}$ entre campos confirmados en controles consecutivos. Si el control se realiza cada N pasos, D abarca esos N pasos, no un cuadro gráfico ni medio barrido GS. El primer control se compara con la condición inicial. El cuadro 1 define las nueve opciones. Para medidas matriciales se hace $D = 0$ fuera de la selección.

Las ocho medidas ΔT se comparan con $a_\Delta + \varepsilon_\Delta s$, en K, con escala s calculada sobre $T - T_{ref}$. Para traza y media, la escala usa valores absolutos y no una suma con signo. Las normas vectoriales y matriciales se distinguen según sus definiciones [7]. Traza y cambio de media no son normas: cambios opuestos pueden cancelarse. La traza ignora variaciones fuera de la diagonal y en una grilla rectangular se denomina suma diagonal. Una diagonal sin nodos libres no confirma el criterio.

Se puede observar todo F o una muestra fija sin reemplazo. Una permutación parcial Fisher–Yates con generador congruencial de 32 bits y semilla registrada reproduce la selección entre implementaciones; no se vuelve a sortear en cada control. La muestra puede omitir regiones relevantes y no proporciona garantía de cobertura. La auditoría física completa permanece activa incluso con muestreo. Por ello, la muestra tiene finalidad comparativa y no elimina el costo de auditar el dominio completo.

El criterio elegido y la auditoría tienen contadores independientes. Se distinguen `converged` (ambos confirmados), `criterion_met` (criterio de cambio satisfecho sin confirmar el equilibrio

físico) y `max_steps` (presupuesto agotado). Un paso temporal diminuto puede producir $D \simeq \Delta t r$ pequeño con r grande. Esta separación evita identificar automáticamente inmovilidad aparente y estacionariedad. Ninguna de estas condiciones equivale por sí sola a una cota del error de temperatura frente al problema continuo.

5. Implementación y trazabilidad

La edición Linux utiliza Python, NumPy, PyOpenCL y una interfaz Tkinter/OpenGL; la edición macOS utiliza SwiftUI, AppKit y Metal, con representación de superficie mediante SceneKit. NumPy calcula en float64; OpenCL, CPU Swift y GPU Metal usan campos float32. Los controles en el host Metal acumulan en Double, lo que no recupera la precisión perdida al actualizar el campo float32.

La escena JSON v1/v2 conserva geometría y asignaciones. El cómputo produce un campo confirmado, separado de las solicitudes del editor y del campo pendiente de publicación. El cambio de procesador reinicia y detiene la corrida. La captura y las isotermas corresponden al campo publicado; los niveles de isotermas están equiespaciados en temperatura, no necesariamente en distancia espacial. Un nivel puede producir varias curvas conexas. La representación 3D es una superficie del campo bidimensional, no una solución tridimensional de conducción.

La exportación guarda campo CSV, escenario, metadatos e historial de hasta 512 controles, con indicación de truncamiento. Los metadatos identifican procesador solicitado y efectivo, pasos o iteraciones, tolerancias, residuos y motivo de parada. En OpenCL se conserva el campo de control anterior en GPU y se leen 15 escalares por comprobación; no se atribuye esa ruta de transferencia al solver Metal. La coincidencia conceptual de formatos no implica identidad binaria o de operaciones aritméticas.

Las campañas analizadas son desarrollos posteriores a Linux 0.2.4 y Metal 0.11.0, sobre un árbol modificado respecto de su commit base. La candidata de distribución se identifica como Linux 0.2.5 y Metal 0.12.0. Esos números posteriores no deben confundirse con una ejecución de instaladores finales ya aceptados. Los cambios de metadatos de empaquetado posteriores a los tests se separan de las fuentes numéricas ensayadas.

6. Materiales y metodología de verificación

6.1. Equipos y precisión

La campaña Linux se ejecutó en AMD Ryzen 9 5900X, Linux 6.12.95, Python 3.11.2 y NumPy 1.26.4, con NVIDIA RTX A2000 como dispositivo OpenCL local de índice 1. Se registraron controlador NVIDIA 570.86.10, OpenCL 3.0 CUDA y PyOpenCL 2026.1.2. El índice identifica ese inventario y debe consultarse de nuevo en otra máquina.

La campaña macOS utilizó Apple M1 con GPU de siete núcleos y 8 GiB de memoria unificada, macOS 15.4.1 (24E263), Xcode 16.4 y Swift 6.1.2. Las mediciones M1 se realizaron en batería; se registró un nivel de 74 %, sin aislamiento de carga ni medición de potencia. Estas condiciones limitan el análisis de rendimiento y no permiten concluir eficiencia energética. Los números de versión corresponden a los registros de las campañas, no a requisitos mínimos del software.

6.2. Referencias y separación de errores

Se utilizaron referencias analíticas, soluciones semidiscretas y balances independientes del criterio de parada. Para un modo senoidal rectangular homogéneo,

$$u(x, y, t) = \sin(\pi x/L_x)\sin(\pi y/L_y)e^{-\lambda t}, \qquad \lambda = \alpha\left[(\pi/L_x)^2 + (\pi/L_y)^2\right]. \tag{11}$$

El autovalor discreto correspondiente es

$$\lambda_h = 4\alpha \left[\frac{\sin^2(\pi \Delta x/(2L_x))}{\Delta x^2} + \frac{\sin^2(\pi \Delta y/(2L_y))}{\Delta y^2} \right]. \tag{12}$$

En estos ensayos se utilizan $k = 1$ W/(m K), $C = 10$ J/(m^3 K), amplitud inicial de 1 K y bordes Dirichlet homogéneos. Son parámetros de verificación, sin ajuste a un experimento físico. El refinamiento temporal se compara con $e^{-\lambda_h t}$ para evitar mezclar error espacial y temporal. Se usan $\Delta t = 0.04, 0.02, 0.01$ s hasta $t = 0.4$ s, en grilla 9×7. El refinamiento espacial usa la solución continua en grillas 17×9, 33×17 y 65×33 hasta $t = 0.02$ s; Euler reduce Δt aproximadamente como h^2 y CN utiliza $\Delta t = 0.002$ s. Los órdenes observados se calculan como

$$p = \frac{\log(e_h/e_{h/2})}{\log 2}, \qquad e = \|T_{num} - T_{ref}\|_\infty . \tag{13}$$

Son estudios de tres niveles en casos suaves. No se calcula un índice GCI ni una incertidumbre general para cualquier escenario; el orden observado no se traslada automáticamente a geometrías recortadas [6].

Los ensayos estacionarios incluyen un perfil parabólico con fuente, una interfaz de dos materiales con resistencias en serie y un perfil lineal con contornos Neumann/Robin. La interfaz se coloca a mitad entre nodos para aislar el operador del desplazamiento de la interfaz. En un círculo con fuente uniforme y borde Dirichlet se compara con $T(r) = T_b + Q(R^2 - r^2)/(4k)$. Se ensayan conservación en dominios rectangulares y elípticos, heterogeneidad y fuentes. En sistemas aislados la energía discreta $E = \sum_i C_i V_i T_i$ debe variar según el aporte de fuentes; con Neumann/Robin se integra el flujo saliente. Euler utiliza flujo inicial y CN su promedio temporal, acorde con la discretización.

6.3. Tolerancias declaradas

En Linux, los sistemas CN usan $a_{CN} = 10^{-13}$ K, $\varepsilon_{CN} = 10^{-12}$ en NumPy y 10^{-8} K, 10^{-6} en OpenCL. Los perfiles estacionarios se controlan con residuo absoluto 10^{-10} K/s y 2×10^{-5} K/s, respectivamente, con tolerancia relativa nula. Además se controla error de temperatura (2×10^{-8} K y 2×10^{-4} K) y flujo de interfaz (10^{-7} y 5×10^{-4} W/m^2). Estos son umbrales de esos ensayos, no constantes físicas ni precisión garantizada del programa.

El refinamiento circular complementario M1 declara residuo 10^{-4} K/s; no sustituye el ensayo original de 33^2 nodos con 5×10^{-5} K/s. Los límites específicos y parámetros completos permanecen en JSON, logs y fuentes de test. También se fuerza un residuo objetivo de 10^{-10} K/s en float32 para comprobar que el estancamiento se informa como no convergencia. No se relajan umbrales silenciosamente para convertir una salida de límite en éxito.

6.4. Protocolo de costo

El benchmark M1 comprende $5 \times 4 \times 2 \times 3 = 120$ mediciones: tamaños 17^2, 33^2, 65^2, 129^2 y 257^2; cuatro métodos; dos procesadores; tres repeticiones. Se compila con optimización `-O`, se calientan cuatro pasos/barridos y se miden 64. Los escenarios de costo usan caja unitaria, $k = 1$, $C = 10$, amplitud senoidal de 1 K y borde de temperatura cero; Euler utiliza $0.8\,\Delta t_{lim}$ y CN $8\,\Delta t_{lim}$. Se excluyen inicialización y compilación de kernels; en Metal se incluyen despacho, carga y lectura del campo, y en CN todas sus iteraciones internas. No se incluyen interfaz, escritura CSV ni auditorías de estacionario exteriores al solver CN. Se informa mediana por combinación; tres repeticiones no caracterizan por sí solas una distribución estadística de rendimiento.

Se compara igual conteo fijo dentro de cada método, no igual tiempo físico entre integradores ni igual error final entre métodos. Un barrido GS incluye ambos colores. El benchmark Linux emplea otra ruta de transferencia; se conservan sus datos pero no se utiliza como una comparación controlada A2000–M1.

Cuadro 2: Evidencia de la campaña. Los renglones se solapan y no se suman como tamaño muestral independiente.

Registro	Resultado o contenido
Suite Linux, 29/10	155 aprobadas, 2 omitidas; 157 descubiertas, sin fallos.
Campaña física Linux	217 comprobaciones aprobadas.
Comparación de criterios Linux	64 casos comparativos, con errores al equilibrio.
Suite M1 final	103 XCTest aprobados; cero fallos y omisiones.
Datos numéricos M1	60 registros NUMERIC.
Campaña complementaria M1	158 registros: 144 criterios, 6 círculos, 4 factores modales y 4 balances.
Exportaciones M1	144 conjuntos campo/escenario/metadatos/historial auditados.
Costo M1	120 tiempos; 8 pares de campos guardados para cotejo.

Cuadro 3: Órdenes observados entre los tres niveles de refinamiento. Cada celda muestra los dos órdenes sucesivos.

Implementación	Euler temporal	CN temporal	Euler espacial	CN espacial
NumPy	1.0275 / 1.0135	2.0008 / 2.0002	2.0095 / 1.9996	1.9966 / 2.0042
OpenCL	1.0275 / 1.0135	2.0011 / 2.0159	2.0112 / 2.0730	1.9970 / 2.0026
Swift	1.0275 / 1.0135	2.0011 / 2.0159	2.0047 / 1.9456	1.9953 / 1.9795
Metal	1.0275 / 1.0135	2.0011 / 2.0159	2.0065 / 1.9410	1.9953 / 1.9820

7. Resultados

7.1. Pruebas, conservación y órdenes observados

El cuadro 2 separa suites y experimentos para evitar sumar controles de diferente naturaleza. En Linux las dos omisiones corresponden a clic nativo con el escritorio bloqueado, no a pruebas numéricas GPU omitidas. En M1 los 103 casos XCTest se aprobaron sin fallos ni omisiones. La revisión posterior en Linux recalculó integridad y métricas de la evidencia recibida; no ejecutó Metal en Linux ni constituye una reproducción independiente en un segundo M1.

El máximo error absoluto de los cuatro balances transitorios independientes Neumann/Robin M1 fue 7.88148×10^{-8} J/m, frente al límite 2×10^{-6} J/m. La cancelación de intercambios internos y la integración consistente de fuentes y contornos se comprobaron también en la campaña Linux. La conservación es una propiedad del problema discreto ensayado: no garantiza que la frontera representada coincida con la geometría física deseada.

Los órdenes del cuadro 3 son compatibles con Euler de orden uno y CN de orden dos en el rango estudiado. Los órdenes espaciales próximos a dos corresponden al rectángulo suave. La figura 2 permite inspeccionar las magnitudes originales de error, además del orden calculado. Las desviaciones de float32 son esperables cuando disminuye el error de truncamiento y cobra importancia el redondeo.

Para el modo de alta frecuencia $z = 4$, Swift obtuvo $g_{CN} = -0.333333254$ y Metal -0.333333373; ambos registraron $g_R = 0.111111104$. El acuerdo con la ecuación (8) muestra que las oscilaciones de CN pueden ser propias del método y no un error de dibujo. La amortiguación inicial no justifica usar pasos arbitrariamente grandes sin estudiar precisión temporal.

7.2. Parada, muestreo y coherencia de exportación

Los 144 casos M1 combinan cuatro métodos, dos procesadores, dos selecciones de nodos y nueve criterios. Se registraron 72 salidas `converged` y 72 `max_steps`. El presupuesto deliberadamente corto fue de 64 pasos; estas últimas no son 72 pruebas fallidas ni prueban divergencia. La batería verifica que el motivo de parada refleje los contadores y el presupuesto. Ensayos separados de paso

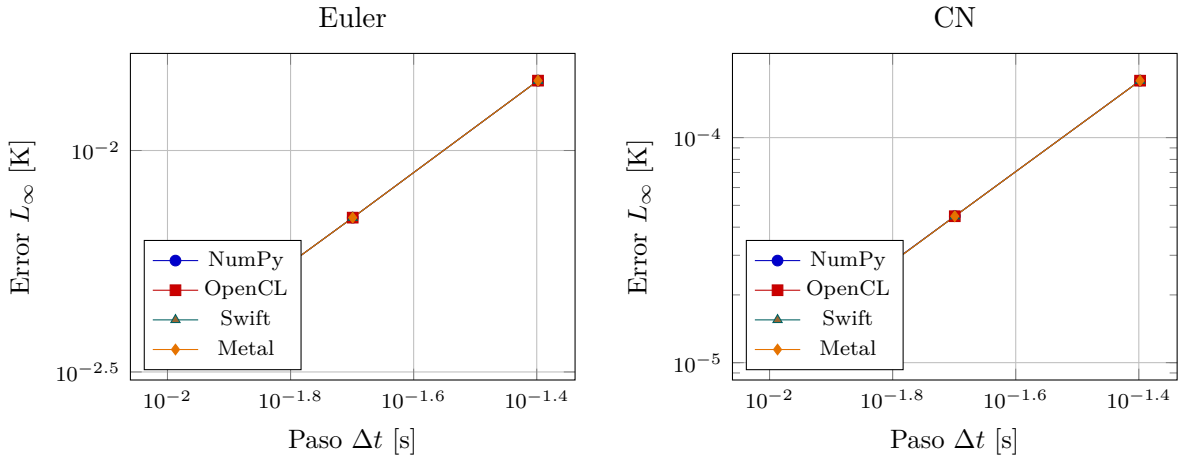


Figura 2: Refinamiento temporal frente al modo semidiscreto, 9×7 nodos, $t = 0.4$ s. Las curvas CPU/GPU casi se superponen; no representan ocho réplicas independientes. Los puntos son errores registrados, sin ajuste.

diminuto exigen `criterion_met` cuando la variación se estabiliza antes que el residuo físico.

La auditoría de los 144 conjuntos verificó dimensiones del campo, coordenadas, temperaturas finitas, procesador solicitado/efectivo, último paso e historial, y coherencia con el escenario. Los métodos estacionarios no atribuyen segundos físicos a sus iteraciones. Esta evidencia acredita contratos de exportación en los casos ensayados, no una validación exhaustiva de toda combinación posible de escenas o acciones del usuario.

7.3. Precisión entre implementaciones

En las 120 mediciones M1, el máximo de las diferencias Swift/Metal registradas a igual conteo fue 1.54972×10^{-6} K. A partir de ocho pares de CSV guardados, para cuatro métodos y tamaños 17^2 y 65^2, la reconstrucción float32 dio máximo 1.49012×10^{-6} K. Son conjuntos distintos, por lo que no se espera que sus máximos coincidan. No se afirma igualdad bit a bit.

Se cotejaron 48 pares Linux/M1 de errores frente a sus referencias analíticas. Ese cotejo compara errores, no directamente campos A2000 contra M1. Tampoco demuestra independencia total: las implementaciones comparten formulación, escenarios y parte del diseño de pruebas. En OpenCL se observó estancamiento de algunos perfiles cerca de 10^{-6} K/s; un caso mixto GS se estabilizó alrededor de 2.27×10^{-6} K/s. Una tolerancia incompatible con float32 no mejora el campo y debe producir una salida informativa.

7.4. Geometría circular y sesgo Robin

Para $R = 0.31$ m, $k = 1$ W/(m K), $Q = 4$ W/m^3 y $T_b = 0$, los errores máximos Metal del círculo Dirichlet fueron 0.0129054, 0.00603398 y 0.00269434 K con 17^2, 33^2 y 65^2 nodos. El refinamiento mejora ese caso, sin establecer un orden universal para dominios curvos.

El ensayo circular Robin evidencia una limitación más persistente (cuadro 4). El círculo se centra en una caja de 0.5×0.5 m, con radio $R = 0.2$ m, $h_b = 15$ W/(m^2 K) y $k = 401$ W/(m K). Se mide la conductancia convectiva discreta en el contorno escalonado y se compara con $h_b 2\pi R$ por unidad de espesor. Las unidades son W/(m K); si se toma espesor de 1 m, el valor numérico coincide con la conductancia en W/K. La comparación no es el problema de resistencia radial total entre centro y ambiente: caracteriza la representación de la frontera.

La aproximación por caras alineadas mide una longitud de tipo Manhattan. Para un círculo de radio R, la suma límite de proyecciones horizontal y vertical es $8R$, frente a $2\pi R$. Si la resistencia

Cuadro 4: Conductancia Robin circular por unidad de espesor, en W/(m K). El campo de origen denomina W/K a la conductancia para espesor unitario.

Grilla	G_h	G_{ideal}	Exceso [%]
41^2	24.744215	18.849556	31.272
81^2	24.372151	18.849556	29.298
161^2	24.186086	18.849556	28.311

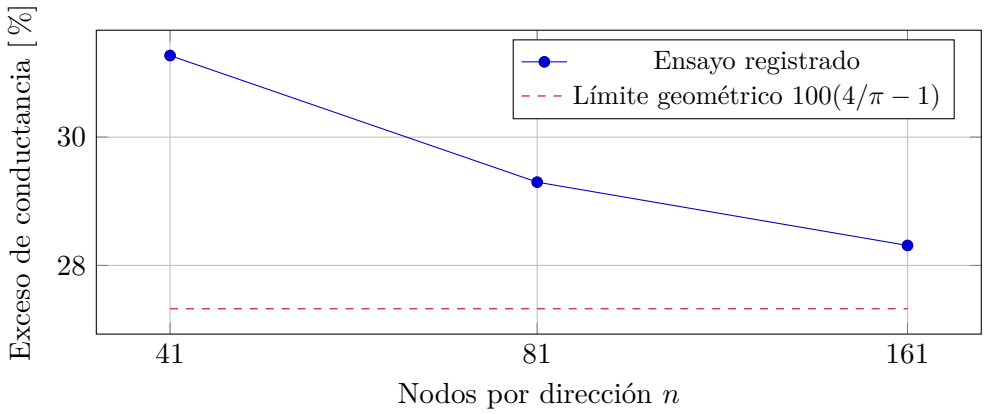


Figura 3: Refinar reduce parte del error de frontera pero no el exceso límite de perímetro escalonado. La línea horizontal es la predicción geométrica, no un ajuste a los tres puntos.

de media celda tiende a cero al refinar, la ecuación (4) conserva el factor geométrico

$$\frac{8R}{2\pi R} - 1 = \frac{4}{\pi} - 1 \simeq 27.32395\,\%. \tag{14}$$

El error observado de 28,31 % en 161^2 nodos es compatible con ese límite; tres grillas no constituyen una demostración numérica general de toda secuencia de máscaras. El argumento geométrico explica por qué aumentar nodos no elimina el sesgo. La prueba correspondiente se aprueba al detectar esta limitación, no al certificar exactitud del círculo ideal. Su solución requiere métricas de contorno apropiadas, no solo tolerancias más estrictas del solver.

7.5. Costo y tamaño del problema

En 257^2 nodos, los cocientes de tiempos medianos Swift/Metal fueron 7,33 (Euler), 5,12 (CN), 13,99 (Jacobi) y 8,09 (GS). Para tamaños pequeños se observan casos en que Swift es más rápido (figura 4). El despacho, sincronización y movimiento de datos pueden dominar cuando el trabajo por lote es pequeño. CN incluye numerosos barridos internos y no debe compararse con un paso Euler como si ambos tuvieran el mismo costo o precisión.

Cada aplicación del stencil requiere trabajo y almacenamiento proporcionales al número de nodos N para una cantidad fija de buffers. Para una región 2D de tamaño físico fijo y grilla

Cuadro 5: Tiempo mediano de 64 pasos/barridos en 257^2 nodos, tres repeticiones. El cociente compara procesadores dentro de cada método, no eficiencia entre métodos.

Método	Swift [s]	Metal [s]	Swift/Metal
Euler	0.016355	0.002231	7.33
CN	1.793044	0.350358	5.12
Jacobi	0.035471	0.002535	13.99
GS	0.041021	0.005071	8.09

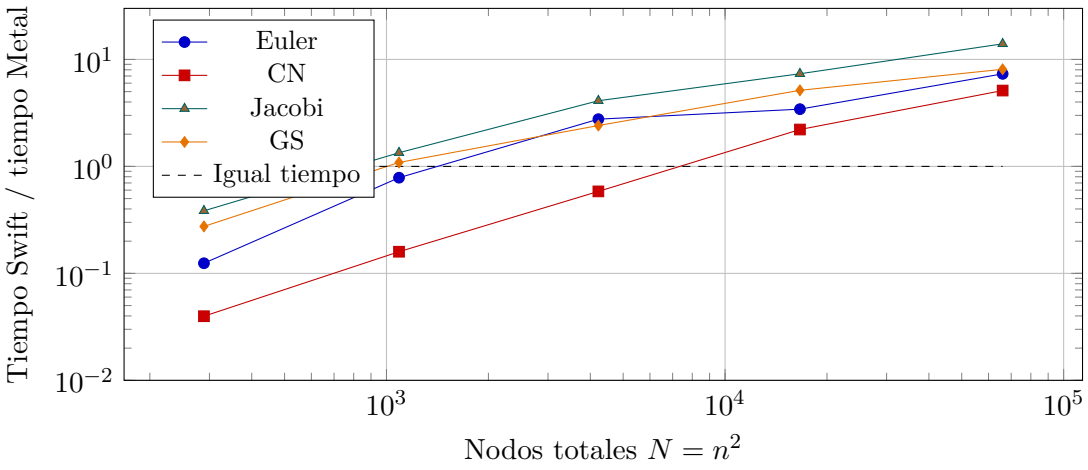


Figura 4: Cocientes de medianas en M1 para 64 pasos/barridos. Valores mayores que uno favorecen Metal; menores que uno favorecen Swift. Se incluyen transferencias de campo Metal; no se mide tiempo hasta igual error ni se compara A2000 con M1.

aproximadamente isotrópica, $N = O(h^{-2})$; el límite explícito $\Delta t = O(h^2)$ lleva a $O(h^{-4}) = O(N^2)$ operaciones para alcanzar tiempo físico fijo, si el costo por aplicación es lineal. Es un argumento asintótico, no un ajuste de los tiempos de esta campaña. En los métodos estacionarios el número de barridos depende de la convergencia; en CN depende además del solver interno. No se midió aquí un consumo máximo de memoria ni energía eléctrica que permita dar cotas prácticas para todos los equipos.

8. Discusión: una herramienta útil para el laboratorio docente

La finalidad educativa del entorno es permitir que un estudiante construya un caso, cambie una condición y observe la evolución sin depender de un equipo de altas prestaciones. El renderizado cumple un papel concreto: hace legible un campo con pocos nodos, permite alternar mapa, superficie y flujo y conserva la posibilidad de animación cuando el cálculo se realiza en CPU. La visualización continua facilita seguir el desplazamiento de zonas calientes y la aproximación al equilibrio, mientras los nodos siguen siendo las incógnitas de un modelo térmico definido.

Esta separación ofrece dos decisiones distintas al docente. Puede mantener una grilla reducida para explorar tendencias y reservar el refinamiento para una comparación de precisión. También puede ampliar el área de dibujo o cambiar la cámara sin modificar el problema numérico. De ese modo, presentar mejor la solución no exige por sí solo aumentar el costo del solver. El beneficio interesa especialmente en laboratorios con equipos reutilizados, donde una GPU dedicada no está disponible en cada puesto.

La herramienta conserva balances de energía, conductividad heterogénea, fuentes y condiciones térmicas de contorno, junto con métodos temporales y estacionarios. Esa base permite proponer actividades que van más allá de una animación ilustrativa. En una primera experiencia puede estudiarse un perfil suave en una grilla reducida, exportar temperaturas y comparar con la solución de referencia. Luego se repite el ejercicio con más nodos y se observa qué cambia en el campo y en el costo. La interpolación de pantalla mantiene una presentación legible en ambos casos; la comparación numérica decide si la grilla inicial era suficiente.

Otra experiencia modifica fuente, material o contorno y utiliza las superposiciones de flujo e isotermas para interpretar el intercambio térmico. Los niveles de isotermas están equiespaciados en temperatura, no en distancia: su agrupamiento puede expresar gradientes intensos. La superficie 3D ayuda a leer el campo bidimensional, pero su altura y relieve son recursos de representación.

No constituyen un problema de conducción tridimensional ni una medición de altura física.

Los controles de parada sirven para que una animación aparentemente inmóvil no se confunda con equilibrio. Una muestra de puntos, la traza o la media pueden estabilizarse mientras persisten variaciones locales. La auditoría residual permite inspeccionar esa diferencia sin abandonar el entorno de trabajo. Del mismo modo, el ensayo de alta frecuencia de CN permite interpretar cambios de signo del método y el refinamiento circular muestra un error geométrico que el renderizado no corrige.

Los datos disponibles respaldan los cálculos ensayados y muestran que para tamaños pequeños el cálculo CPU puede resultar conveniente. La hipótesis de acceso a equipos modestos se apoya en esa arquitectura y en las pruebas de uso comunicadas, pero requiere una campaña específica de rendimiento interactivo para cuantificarla. Los tiempos de 64 pasos no permiten inferir cuadros por segundo de una ventana con isotermas y superficie 3D. Tampoco se midieron resultados de aprendizaje con estudiantes: las actividades descritas son propuestas de uso, cuya utilidad pedagógica debe evaluarse en su contexto.

9. Limitaciones y desarrollo futuro

Los ejemplos de bajo costo requieren una grilla suficiente para el fenómeno observado. En particular, el uso de pocos nodos no asegura representar fuentes delgadas o contornos curvos con precisión. La implementación actual trabaja sobre grillas cartesianas estructuradas. Curvas y materiales se rasterizan; el sesgo Robin circular es una restricción de uso explícita. No se dispone de mallas no estructuradas, fronteras curvilíneas ajustadas, radiación ni propiedades dependientes de temperatura. Las componentes estacionarias sin referencia requieren un tratamiento posterior del espacio nulo. La resolución iterativa y float32 limitan la tolerancia alcanzable.

Como siguiente etapa se propone discretizar conducción en coordenadas polares planas para geometrías circulares, con tratamiento del origen, periodicidad angular, áreas y normales consistentes y nuevos ensayos de conservación y orden. Una grilla polar puede seguir siendo estructurada; no debe denominarse automáticamente no estructurada. Las mallas no estructuradas para geometrías generales constituyen otra línea de desarrollo. El resultado Robin obliga a verificar métricas geométricas antes de inferir mejoras a partir del número de nodos.

La aceptación manual del editor y de la interfaz M1, la primera apertura en otra Mac, Gatekeeper y la instalación/desinstalación final del paquete Debian/Devuan no quedaron cerradas por las campañas numéricas. La app recibida tiene firma local ad hoc, sin Developer ID ni notarización. Dos tests Linux de clic nativo permanecen omitidos por la sesión bloqueada. Estas tareas afectan la aceptación de distribución, aunque no invalidan por sí mismas las mediciones numéricas documentadas. No se declara una distribución de producción plenamente verificada.

10. Conclusiones

Se implementó un entorno educativo que separa la resolución de la ecuación de calor de la resolución de su imagen. OpenGL permite presentar un campo calculado con pocos nodos mediante mapas y superficies animadas; el tamaño de la ventana no obliga a aumentar la cantidad de incógnitas. Esta aplicación del renderizado permite orientar el trabajo numérico hacia la física que el ejercicio necesita resolver y mantener una ruta de cálculo CPU para equipos sin GPU dedicada de cómputo.

La herramienta integra edición de escenarios, materiales, fuentes, contornos, métodos temporales y estacionarios y exportación de resultados. Las verificaciones aportan una base para interpretar sus imágenes como representaciones de un modelo conservativo, dentro de los casos y tolerancias estudiados. La suavidad visual permanece separada de la exactitud: se requiere refinamiento

cuando lo demanda el problema, y el sesgo Robin de un contorno circular escalonado exige corregir la geometría, no el dibujo.

El bajo costo se plantea como una decisión de implementación y de uso de grillas reducidas, con antecedentes de funcionamiento en equipos de laboratorio modestos. Para caracterizarlo como prestación de hardware deben medirse tasa de cuadros, latencia y memoria en esos equipos, con escenas y precisión declaradas. El resultado actual es una herramienta abierta para explorar fenómenos térmicos con recursos acotados, acompañada de evidencia numérica que permite discutir sus alcances y continuar su mejora.

Disponibilidad de código, datos y reproducibilidad

El repositorio del proyecto es `https://github.com/javoxa/YoguiCuentas`. El código y la documentación del proyecto se distribuyen bajo Apache License 2.0, con el alcance indicado en `LICENSE` y `NOTICE`; este trabajo no amplía derechos sobre imágenes ni material de terceros. Las referencias bibliográficas conservan sus propios derechos.

La evidencia analizada se identifica por los manifiestos y rutas del apéndice. La candidata local y este manuscrito aún no constituyen una publicación depositada con DOI; la disponibilidad pública de la revisión concreta debe comprobarse al depositar el artículo. Un commit base de un árbol modificado no identifica por sí solo todos los archivos ensayados. Para una versión de archivo se recomienda depositar conjuntamente fuentes, datos, manifiestos y este manuscrito con un identificador persistente, sin atribuirle retrospectivamente un DOI inexistente.

Autoría, financiación y declaraciones

Autor de correspondencia: Javier Armando Gutierrez, Departamento de Física, FCE, Universidad Nacional de Salta, Salta, Argentina; g.javier.ar@gmail.com. El autor de correspondencia será responsable de la revisión final y del envío. No se proporcionó una declaración de financiación ni de conflictos de interés; no se sustituye esa ausencia por una declaración de inexistencia. Este manuscrito se preparó con asistencia de una herramienta de inteligencia artificial para inspección de artefactos, organización, redacción y generación de figuras desde datos registrados; los autores humanos deben revisar y asumir responsabilidad por el contenido, los resultados y la declaración exigida por el destino de publicación. No se asigna autoría a la herramienta.

A. Identificación de la evidencia y reproducción

La campaña Linux del 29/10 registra árbol modificado respecto del commit base

`58742d505561a9e75466dca0ec0f748e0b285dc0.`

El hash conjunto de fuentes numéricas es

`e576c94fc46a53d1efabf29c9b291d2569dd7580370554cb8f50788dda384c78.`

En la devolución M1 se verificaron 71/71 archivos finales, 32/32 entradas del manifiesto de tests, 18/18 del benchmark y 1374/1374 archivos de entrega. La selección pública conserva 635 archivos con manifiesto propio. El binario recibido coincide con

`22ae2a2609a65878a755ebc0b64570f4e3b8c9c6581096444e6af9b2e2d9237d.`

La firma se acredita por el log macOS recibido; en Linux se verificó el hash, no se reejecutó `codesign`. Después se modificó únicamente la versión predeterminada del script de empaquetado

Metal a 0.12.0; las fuentes Swift de cálculo y los tests integrados conservan los bytes ensayados. La trazabilidad documental no equivale a una nueva ejecución del instalador.

Cuadro 6: Ubicación relativa de los artefactos principales en la raíz del repositorio.

Artefacto	Ruta
Linux, metadatos y métricas	`verification/2026-10-29/numerical/report.json`
Linux, suite completa	`verification/2026-10-29/unittest.log`
Criterios Linux	`verification/2026-10-29/criteria/README.md`
M1, suite final	`verification/2026-10-29/m1/final-02/swift-test.log`
M1, mediciones y escenarios	`verification/2026-10-29/m1/final-02/`
M1, costo	`verification/2026-10-29/m1/benchmark-final/timings.csv`
Auditoría de devolución	`verification/2026-10-29/m1-review/audit.json`
Manifiesto de evidencia M1	`verification/2026-10-29/m1/MANIFIESTO_EVIDENCIA.sha256`
Datos de tablas/figuras	`technical_report/datos_manuscrito.json`

Para ejecutar la campaña Linux sin interfaz:

```
python -m yoguicuentas.verification -backend numpy
  -skip-benchmark -output resultados/verificacion
python -m yoguicuentas.verification -backend opencl -device INDICE
  -skip-benchmark -output resultados/verificacion-gpu
```

Cada comando ocupa dos líneas tipográficas: debe ejecutarse como una sola línea. El índice se obtiene con `python -m yoguicuentas -list-devices`. Pedir OpenCL sin dispositivo disponible debe producir un fallo explícito. Para la campaña completa M1, desde la raíz:

```
bash metal/yoguicuentas_metal/verificar_numerica.sh
  verification/repeticion full
```

La orden anterior también debe unirse en una sola línea. La ejecución requiere herramientas Swift y una Mac con Metal para no omitir GPU; una app descargada no incorpora el entorno XCTest. Los logs deben conservar también fallos y omisiones. Para regenerar los datos y segmentos de figuras del manuscrito se proporciona `technical_report/preparar_datos.py`; los números provienen de los registros indicados, sin datos sintéticos. El archivo LaTeX es autónomo, con bibliografía y coordenadas incluidas.

B. Resumen completo del costo M1

Cuadro 7: Medianas de los 120 tiempos M1, agrupados en 40 combinaciones tamaño/método/procesador. Los escenarios y las repeticiones individuales se conservan con la evidencia.

$n_x = n_y$	Método	Swift [s]	Metal [s]	Swift/Metal
17	Euler	0.0000654	0.0005262	0.124
17	CN	0.0074421	0.1871517	0.040
17	Jacobi	0.0002719	0.0007073	0.384
17	GS	0.0003767	0.0013710	0.275
33	Euler	0.0006045	0.0007709	0.784
33	CN	0.0368816	0.2314394	0.159
33	Jacobi	0.0013004	0.0009687	1.342
33	GS	0.0016968	0.0015590	1.088
65	Euler	0.0023663	0.0008560	2.764
65	CN	0.1347383	0.2307876	0.584
65	Jacobi	0.0031762	0.0007729	4.109
65	GS	0.0030916	0.0012828	2.410
129	Euler	0.0040920	0.0011935	3.429
129	CN	0.5324523	0.2407363	2.212
129	Jacobi	0.0072743	0.0009905	7.344
129	GS	0.0097022	0.0018866	5.143
257	Euler	0.0163547	0.0022308	7.331
257	CN	1.7930442	0.3503582	5.118
257	Jacobi	0.0354705	0.0025353	13.990
257	GS	0.0410212	0.0050712	8.089

Referencias

[1] J. Kessenich / The Khronos Group. *The OpenGL Shading Language*, versión 1.40, revisión 08, 22/11/2009. Especificación oficial; apartados de entradas/salidas e interpolación. https://registry.khronos.org/OpenGL/specs/gl/GLSLangSpec.1.40.pdf. Consultado el 02/11/2026.

[2] J. Crank y P. Nicolson. A practical method for numerical evaluation of solutions of partial differential equations of the heat-conduction type. *Proceedings of the Cambridge Philosophical Society*, 43(1), 50–67, 1947. doi:10.1017/S0305004100023197. Reimpresión: *Advances in Computational Mathematics*, 6, 207–226, 1996, doi:10.1007/BF02127704.

[3] R. Rannacher. Finite element solution of diffusion problems with irregular data. *Numerische Mathematik*, 43, 309–327, 1984. doi:10.1007/BF01390130.

[4] R. Barrett, M. Berry, T. F. Chan, J. Demmel, J. M. Donato, J. Dongarra, V. Eijkhout, R. Pozo, C. Romine y H. van der Vorst. *Templates for the Solution of Linear Systems: Building Blocks for Iterative Methods*. SIAM, 1994. https://netlib.org/templates/templates.html.

[5] NPARC Alliance / NASA Glenn Research Center. *Verification Assessment*. Tutorial de verificación y validación. https://www.grc.nasa.gov/www/wind/valid/tutorial/verassess.html. Consultado el 30/10/2026.

[6] NPARC Alliance / NASA Glenn Research Center. *Examining Spatial (Grid) Convergence*. Tutorial de verificación y validación. https://www.grc.nasa.gov/www/wind/valid/tutorial/spatconv.html. Consultado el 30/10/2026.

[7] NumPy Developers. *numpy.linalg.norm*: normas vectoriales y matriciales. Documentación oficial. https://numpy.org/doc/stable/reference/generated/numpy.linalg.norm.html. Consultado el 30/10/2026.

[8] Proyecto YoguiCuentas. *Fuentes, escenarios y evidencia numérica de YoguiCuentas*. Campañas del 28–29/10/2026, desarrollos posteriores a Linux 0.2.4 y Metal 0.11.0. https://github.com/javoxa/YoguiCuentas. Revisión identificada por los manifiestos del apéndice; sin DOI asignado en este manuscrito.